

```

%% Naive Bayes Model
%% Basic NB Model.
% Changing distribution each time

% Below needs to be applied before testing.
XTestBS = table2array(XTestBS);
YTestBS = table2array(YTestBS);
%%

M1 = fitcnb(XTrain1BS,YTrain1BS,"DistributionNames","normal");

rng(1);
cvM1 = crossval(M1);
YpredM1=kfoldPredict(cvM1);

confMatM1 = confusionmat(YTrain1BS,YpredM1);

accuracyM1 = (confMatM1(1,1) + confMatM1(2,2)) / sum(sum(confMatM1))
%%
M2 = fitcnb(XTrain1BS,YTrain1BS,"DistributionNames","normal");

% Cross fold
rng(1);
cvM2 = crossval(M2);
YpredM2=kfoldPredict(cvM2);

confMatM2 = confusionmat(YTrain1BS,YpredM2);

accuracyM2 = (confMatM2(1,1) + confMatM2(2,2)) / sum(sum(confMatM2))
%%
M3 = fitcnb(XTrain1BS,YTrain1BS,"DistributionNames","kernel");

% Cross fold
rng(1);
cvM3 = crossval(M3);
YpredM3=kfoldPredict(cvM3);

confMatM3 = confusionmat(YTrain1BS,YpredM3);

accuracyM3 = (confMatM3(1,1) + confMatM3(2,2)) / sum(sum(confMatM3))
%%
distNames2 =
{'kernel','kernel','kernel','mvmn','mvmn','mvmn','mvmn','mvmn','mvmn','mvmn','mvmn','mvmn'};

M5 = fitcnb(XTrain1BS,YTrain1BS,"DistributionNames",distNames2);

rng(1);
cvM5 = crossval(M5);
YpredM5=kfoldPredict(cvM5);

confMatM5 = confusionmat(YTrain1BS,YpredM5);

accuracyM5 = (confMatM5(1,1) + confMatM5(2,2)) / sum(sum(confMatM5))
%% AUTO function of NB model to find out which one nest fits the model

%https://www.mathworks.com/help/stats/fitcnb.html The auto function will
%select the hyperparameter to optimize the data. Expected-improvement-plus

rng(1)

classNames = {'0','1'};
Mdl = fitcnb(XTrain1BS,YTrain1BS,'ClassNames',\
classNames,'OptimizeHyperparameters','auto',...
'HyperparameterOptimizationOptions',struct('AcquisitionFunctionName',...
'expected-improvement-plus'));

```

```

%%
% Applying the value of the above result into this; whilst applying best distribution
% to each.
%
%
%% Optimised NB model

rng(1);
distNames2 =
{'kernel','kernel','kernel','mvnm','mvnm','mvnm','mvnm','mvnm','mvnm','mvnm','mvnm','mvnm'};
width= 0.31116;
M6 = fitcnb(XTrain1BS,YTrain1BS,"DistributionNames",distNames2,"width",width)
%%
rng(1);
cvM6 = crossval(M6);
YpredM6=kfoldPredict(cvM6);

confMatM6 = confusionmat(YTrain1BS,YpredM6);

accuracyM6 = (confMatM6(1,1) + confMatM6(2,2)) / sum(sum(confMatM6))
%% Performance Measures in NB

NB_Predict = predict(M6,XTestBS);
NB_Result = confusionmat(YTestBS,NB_Predict);

accuracyNB = (NB_Result(1,1) + NB_Result(2,2)) / sum(sum(NB_Result))

precisionNB = NB_Result(1,1) / (NB_Result(1,1) + NB_Result(2,1))

recallNB = NB_Result(1,1) / (NB_Result(1,1) + NB_Result(1,2))

specificityNB = NB_Result(2,2) / (NB_Result(2,2) + NB_Result(2,1))

f1scoreNB = 2 * precisionNB * recallNB / (precisionNB + recallNB)

classNames = {'0','1'};
%% Confusion matrix NB

N = confusionchart(NB_Result)
title('NB in Testing Confusion Matrix');
N.ColumnLabels = {'0','1'};
N.RowLabels = {'0','1'};

% Set the x-axis labels
N.ColumnLabels = {'0','1'};

% Set the y-axis labels
N.RowLabels = {'0','1'};
%%
%
%% Plot ROC curves for tuned NB and LR model performance on held out test data

%
[NB_X,NB_Y,NB_T,NB_AUC] = perfcurve(YTestBS,NB_Predict(:,2),1);
NB_AUC

% LR
[LR_X,LR_Y,LR_T,LR_AUC] = perfcurve(YTestBS,LR_Predict(:,2),1);
LR_AUC

% Reference Line
x_ROC = linspace(0,1,100);
y_ROC = x_ROC;

figure(15)
set(gcf,'color','w');

```

```

plot(x_ROC,y_ROC, '--k', 'HandleVisibility','off')
hold on
plot(NB_X,NB_Y)
plot(LR_X,LR_Y)
legend('NB','LR','Location','Best')
xlabel('False positive rate'); ylabel('True positive rate');
title('ROC Curves for NB and LR in Testing')
hold off
%% % Generate predictions on the test data using LR x NB

[YPred1,Scores1] = predict(M6,XTestBS);
[FPR1,TPR1,T1,AUC1] = perfcurve(YTestBS,Scores1(:,2),1);

[YPred2,Scores2] = predict(L3,XTestBS);
[FPR2,TPR2,T2,AUC2] = perfcurve(YTestBS,Scores2(:,2),1);

% Plot the ROC curves on the same figure
plot(FPR1,TPR1)
hold on
plot(FPR2,TPR2)
hold off
xlabel('False Positive Rate')
ylabel('True Positive Rate')
legend('NB Model','LR Model','Location','Best')
title('ROC curve on testing')
%% AUC

[Ypred,scores] = predict(M6,XTestBS) % Predict the labels and scores for the input
data
[Xroc,Yroc,T,AUC] = perfcurve(YTestBS,scores(:,2),'1')
[Ypred1,scores1] = predict(L3,XTestBS) % Predict the labels and scores for the input
data
[Xroc1,Yroc1,T1,AUC1] = perfcurve(YTestBS,scores1(:,2),'1')
%% AUC --> ROC is the plot of the AUC.

plot(Xroc,Yroc) % Plot the ROC curve
xlabel('False positive rate') % Label the x-axis
ylabel('True positive rate') % Label the y-axis
title('ROC curve for testing data') % Add a title
%%
%
%%

```